ThreatLens

Intelligent Network Intrusion Detection using Classical AI and Machine Learning

Aazan Noor Khuwaja (Roll #24P-0706) · Uzair Shoaib (Roll #24P-0507)

June 5, 2026

Abstract

Network intrusion detection plays a vital role in protecting modern computing infrastructure from malicious cyber attacks. In this project, we developed **ThreatLens**, an automated classification system using a balanced dataset of 6,000 connection records to differentiate between normal and malicious network traffic. We implemented and benchmarked a variety of methodologies, ranging from a heuristic rule-based simple reflex agent to supervised machine learning models including K-Nearest Neighbors, Gaussian Naive Bayes, and Logistic Regression, as well as unsupervised clustering via a custom K-Means implementation and a Genetic Algorithm for feature selection. Our empirical results show that the supervised learning techniques drastically outperform the baseline agent, with the K-Nearest Neighbors classifier (k = 1) achieving our highest classification accuracy of **95.58%**.

1 Introduction

ThreatLens is an intelligent Network Intrusion Detection System (NIDS) that acts as a digital security guard for computer networks. Its primary role is to monitor all incoming and outgoing network traffic continuously, inspecting data packets to identify suspicious or malicious activity. When a pattern matching a known cyber attack is detected, ThreatLens alerts the network administrator so that countermeasures can be taken before real damage occurs.

This problem is of utmost significance because nearly every modern business, healthcare centre, and government system depends entirely on the Internet. Once a malicious actor penetrates a network, they can steal private user data, deploy ransomware, or disrupt critical services. Human operators cannot inspect networks manually at the scale of millions of connections per minute, so there is an urgent need for intelligent, automated systems capable of making accurate real-time decisions.

In this project we aim to design and evaluate a range of network threat identification techniques of increasing sophistication. We begin with a rule-based reflex agent that uses hand-picked thresholds derived from exploratory data analysis. We then train three classical supervised classifiers (K-Nearest Neighbors, Gaussian Naïve Bayes, and Logistic Regression), apply an unsupervised K-Means clustering algorithm, and finally use a custom Genetic Algorithm to perform feature selection. These approaches are compared systematically to determine which strategy best protects digital networks while remaining interpretable and efficient.

2 Dataset

The *network_traffic.csv* dataset contains 6,000 rows of network connection records. It is perfectly balanced, with exactly 3,000 normal traffic samples (label 0) and 3,000 attack samples (label 1), ensuring that no model can gain a trivial accuracy advantage by predicting the majority class. Every connection is described by 15 numeric features capturing measurements such as bytes transferred, connection duration, error rates, and counts of similar concurrent connections. The target column *label* is binary:

- 0 — Normal traffic
- 1 — Attack traffic

A summary of the dataset properties:

- Rows: 6,000 (balanced — 3,000 per class)
- Features: 15 numeric
- Target: label (0 = normal, 1 = attack)
- Missing values: none

3 Task 1: Data Exploration

Method

The dataset was loaded using pandas. We printed the shape, the first few rows with df.head(), and descriptive statistics with df.describe() to understand the range and spread of each feature. We then produced a bar chart of the class distribution and overlaid per-class histograms for three informative features: count, src_bytes, and serror_rate.

Results and Observations

The class distribution confirms a perfectly balanced dataset with exactly 3,000 samples in each class. This is ideal because it makes overall accuracy a reliable evaluation metric and removes the need for class-weight adjustments.

Inspecting df.describe() revealed very large differences in feature scale. Variables such as src_bytes and count can range into the thousands, whereas error-rate features like serror_rate are bounded between 0.0 and 1.0. This disparity means that distance-based algorithms such as KNN and gradient-based methods such as Logistic Regression will require feature standardisation before training.

The per-class histograms expose clear behavioural boundaries. In the count histogram, normal traffic clusters at low connection counts and drops off sharply, whereas attack traffic stretches across a broad range and forms dense clusters at high values — characteristic of automated flood or scan activity. The serror_rate histogram shows that benign traffic flatlines near 0.0, while a large proportion of malicious connections spike at 1.0, indicating failed SYN handshakes or bad protocol requests. These observations directly guided the threshold selection in Task 2.

4 Task 2: Simple Reflex Agent

Method

A simple reflex agent selects an action based solely on the current percept — one connection record — using a fixed set of if–then rules. It has no internal memory and does not consider past or future observations. We implemented a Python function *reflex_agent(row)* that inspects three features chosen from the Task 1 histograms and returns a prediction of 0 (normal) or 1 (attack):

- **If serror_rate > 0.5**, classify as attack (1). *Rationale:* A high SYN-error rate strongly indicates failed handshake attempts typical of port scans or DoS floods.
- **Else if same_srv_rate < 0.4**, classify as attack (1). *Rationale:* A very low proportion of connections to the same service suggests scanning across multiple services.
- **Else if count > 200**, classify as attack (1). *Rationale:* An unusually high connection count is a hallmark of automated attack tools.
- **Otherwise**, classify as normal (0).

The agent was applied to every row in the full 6,000-record dataset using df.apply(reflex_agent, axis=1).

Results

The reflex agent achieved **86.00% accuracy** on the full 6,000-sample dataset, with precision 0.8241, recall 0.9160, and F1-score 0.8676. This is a strong baseline given that the rules are hand-crafted without any training, confirming that the three chosen features are genuinely discriminative.

This is an example of a classic simple reflex agent: it maps a percept (the current row) directly to an action (normal or attack) via fixed condition-action rules, with no internal state or memory of past connections.

5 Task 3: Supervised Machine Learning

Method

The dataset was split once using stratified 80/20 train-test split (random_state=42). All distance-sensitive models (KNN, Logistic Regression) were trained on StandardScaler-normalised features fitted only on the training set. Three classifiers were evaluated:

- **K-Nearest Neighbors (KNN):** $k \in \{1, 3, 5, 7, 9, 11\}$ evaluated; best k chosen by test accuracy.
- **Gaussian Naïve Bayes:** Probabilistic baseline; no scaling required.
- **Logistic Regression:** Linear classifier with max_iter=1000.

Results

Model	Accuracy	Precision	Recall	F1-Score
Gaussian Naïve Bayes	78.75%	0.9320	0.6167	0.7420
Logistic Regression (All Features)	91.83%	0.9022	0.9383	0.9199
KNN (k=1) — Best Classifier	**95.58%**	**0.9458**	**0.9667**	**0.9563**

Table 1: Supervised classifier comparison on the held-out test set. KNN with k=1 is the best classifier, achieving 95.58% accuracy and 0.9563 F1-score. KNN's accuracy peaks at k=1 and monotonically decreases as k increases, confirming that the decision boundaries are highly local

and non-linear.

6 Task 4: Custom K-Means Clustering & 2D PCA

Method

We implemented K-Means (K=2) entirely from scratch in plain Python without using any external clustering library. The algorithm initialises two centroids randomly, assigns each point to the nearest centroid using Euclidean distance, recomputes centroids as the mean of each cluster, and repeats until convergence. All features were scaled using StandardScaler before clustering. Principal Component Analysis (PCA) with n_components=2 was used purely for 2D visualisation of the high-dimensional clusters.

Results

The custom K-Means algorithm converged and achieved approximately **86.60% accuracy** (after resolving the cluster-label flip ambiguity via the confusion matrix). The PCA scatter plots showed clear visual separation between the two clusters, closely mirroring the true label boundaries — confirming that the 15 features naturally encode meaningful discriminative structure even without labels.

7 Task 5: Genetic Algorithm Feature Selection

Method

We designed a custom binary Genetic Algorithm to search for a feature subset that maximises Logistic Regression test accuracy. Each chromosome is a 15-bit binary vector (one bit per feature). The fitness function trains Logistic Regression on the selected features and returns test accuracy. The GA uses:

- Population size: 10 chromosomes
- Generations: 10
- Selection: Roulette Wheel (fitness-proportionate)
- Crossover: Single-Point (rate = 0.8)
- Mutation: Bit-Flip (rate = 0.05 per bit)

Results

Configuration	Features Used	Test Accuracy
Logistic Regression (all features)	15	91.83%
Logistic Regression (GA-selected subset)	10	91.50%

Table 2: Feature selection comparison — full feature set vs. GA-selected 10-feature subset. The accuracy drop of only 0.33 percentage points confirms that 5 of the 15 features contribute minimal discriminative power. The reduced model is simpler and faster to train and predict, which is valuable in a high-throughput ThreatLens deployment where inference latency matters. The result also demonstrates that the GA successfully explored the 2^15 = 32,768 possible feature subsets and found a near-optimal compact solution within just 10 generations.

8 Benchmark Results Summary

Model / Algorithm	Category	Accuracy	F1-Score	Key Finding
Simple Reflex Agent	Rule-Based	86.00%	0.8676	Hand-crafted if-then rules; no training
Gaussian Naïve Bayes	Supervised	78.75%	0.7420	Independence assumption violated
Logistic Regression	Supervised	91.83%	0.9199	Strong linear benchmark; interpretable
K-Means (K=2)	Unsupervised	~86.60%	—	No labels; natural cluster structure
GA + Logistic Reg. (10 feats)	Evolutionary	91.50%	0.9167	33% feature reduction; 0.33% accuracy loss
KNN (k=1) ★ Best	**Supervised**	**95.58%**	**0.9563**	**Non-linear boundaries; highest accuracy**

9 Discussion

When all five approaches are compared, KNN classifier with k = 1 achieved the highest overall test accuracy (95.58%) and the highest F1 score (0.9563), making it the best classifier in this study. It is successful because it is able to learn the complex, non-linear decision boundaries in the 15-dimensional feature space, which distinguishes sophisticated attack patterns from normal traffic.

With three handcrafted rules, the simple reflex agent obtained a very decent accuracy of 86.0%. Its core issue, however, is that it considers each connection individually and applies fixed thresholds, meaning any single connection can be easily evaded if an attacker remains just below any one threshold. It also does not allow for feature interactions.

Gaussian Naïve Bayes was the worst of the three supervised models (78.75%), mostly because of its very low recall rate (61.67%). In a security context, failing to flag more than 33% of actual attacks is a critical weakness. This indicates that the independence assumption does not hold for correlated network traffic features.

After KNN, Logistic Regression (91.83%) provides the best balance between interpretability and performance, offering an explicit linear decision boundary that generalises well to unseen traffic distributions. The unsupervised K-Means correctly clustered the data with ~86.6% accuracy with no labels, validating that the feature space naturally encodes meaningful structure. This makes K-Means useful as a pre-processing or anomaly-detection module even with limited labelled data.

If given more time to expand ThreatLens, we would model the data as a topological graph using Graph Neural Networks (GNNs), representing IP addresses as nodes and connections as edges, to detect distributed, coordinated threats such as botnet activity or multi-vector DDoS attacks by analysing relational network structure.

10 Conclusion

In this project we successfully designed and evaluated ThreatLens, a multi-tiered Network Intrusion Detection System, progressing from a rigid rule-based baseline to supervised machine learning, unsupervised clustering, and evolutionary feature optimisation. Our empirical results clearly demonstrate that data-driven classification algorithms outperform manual heuristics. The optimised K-Nearest Neighbors model (k=1) delivered the highest test accuracy of 95.58% while minimising missed attacks. The Genetic Algorithm showed that the feature space can be reduced by one third with negligible loss in accuracy, supporting leaner real-time deployment. Together, these findings confirm that even classical machine learning techniques, properly implemented and evaluated, can

form a robust foundation for automated network security systems like ThreatLens.

Statement of Contribution

Aazan Noor Khuwaja was responsible for the initial project phases: exploratory data analysis, class-distribution and feature-histogram plots, the rule-based reflex agent, and training and evaluating the three supervised classifiers (Tasks 1, 2, and 3). Uzair Shoaib handled the remaining experimental components: the custom K-Means clustering implementation, PCA visualisation, and the Genetic Algorithm for feature selection (Tasks 4 and 5). Both members collaborated equally on interpreting results, writing the report, and preparing the final submission.